Design and Implementation of a Database Management System for Courier Parcel Booking and Tracking Management

Project 15

Submitted by

Akshaya
Srinidhi
Sahasra

Department of Computer Science and Engineering
Course: Database Management Systems
Academic Year: 2026–2027

Abstract

The Courier Parcel Booking and Tracking Management System is a MySQL-backed database application that records every step of a parcel's life: booking, charge calculation, routing, agent assignment, scanning at branches and hubs, delivery attempts and payment. The database has 13 normalized tables (3NF) linked by primary keys, foreign keys, unique, NOT NULL and CHECK constraints. Business rules are enforced inside the database by three triggers, and management reports are exposed as SQL views. A web simulation lets users book a parcel, add scans, pay, assign agents and run reports against the database. This report covers the design, SQL implementation, testing and simulation of the system.

Index Terms: courier management, parcel tracking, MySQL, relational database, normalization, triggers, views, transactions.

I. Introduction and Problem Statement

Manual registers cannot give reliable end-to-end tracking of parcel booking, routing, scanning, delivery and payment. Customer details, charges, scan history and payments end up in separate places, so records are duplicated, inconsistent and slow to retrieve. A courier company needs one centralized database where a parcel's status is always derived from its recorded events.

This project designs and implements such a database in MySQL, with a web simulation on top of it.

Objectives

- Centralize parcel, booking and payment records.
- Provide real-time tracking through scan events.
- Enforce business rules in the database itself (constraints and triggers).
- Reduce redundancy through normalization up to 3NF.
- Produce management reports with views and queries.

Scope

- Booking, charge calculation and routing.
- Agent assignment, delivery and failed attempts (up to three per parcel).
- Reports: current location, delayed shipments, branch load, route performance, failed attempts and revenue.

Intended Users

Customers, branch staff, delivery agents, and admin / management.

II. System Requirements

The system stores 13 kinds of records and must enforce the rules below.

Entities

Entity	Role in the system
Customer	Person who books parcels
Branch	Office where parcels are booked, picked up and delivered
Hub	Sorting centre on a route

Entity	Role in the system
ServiceType	Standard, Express or Same Day, with its rates and delivery limit
Booking	One shipping request by a customer
Parcel	The item shipped, with a unique tracking number
Route and Vehicle	Hub-to-hub leg and the vehicle that runs it
DeliveryAgent	Staff member who delivers from a branch
ScanEvent	One timestamped step in a parcel's journey
DeliveryAttempt	One delivery try, successful or failed
Charge and Payment	Amount billed and amount paid for a booking

Business Rules Enforced by the Database

- A tracking number, phone number and vehicle registration are each unique.
- Weights, charges, payments and distances must be greater than zero.
- Scans for a parcel are numbered 1, 2, 3 … without gaps, and none are allowed after delivery.
- Total payments for a booking can never exceed its total charges.
- A scan must have a branch or a hub; a parcel has at most 3 delivery attempts.

Software Requirements

Requirement	Technology
Database	MySQL 8.0
Query language	SQL (DDL, DML, views, triggers, transactions)
Application	Web simulation hosted on Render
Client	Any modern web browser
Script	`courier_db.sql`

III. ER Diagram

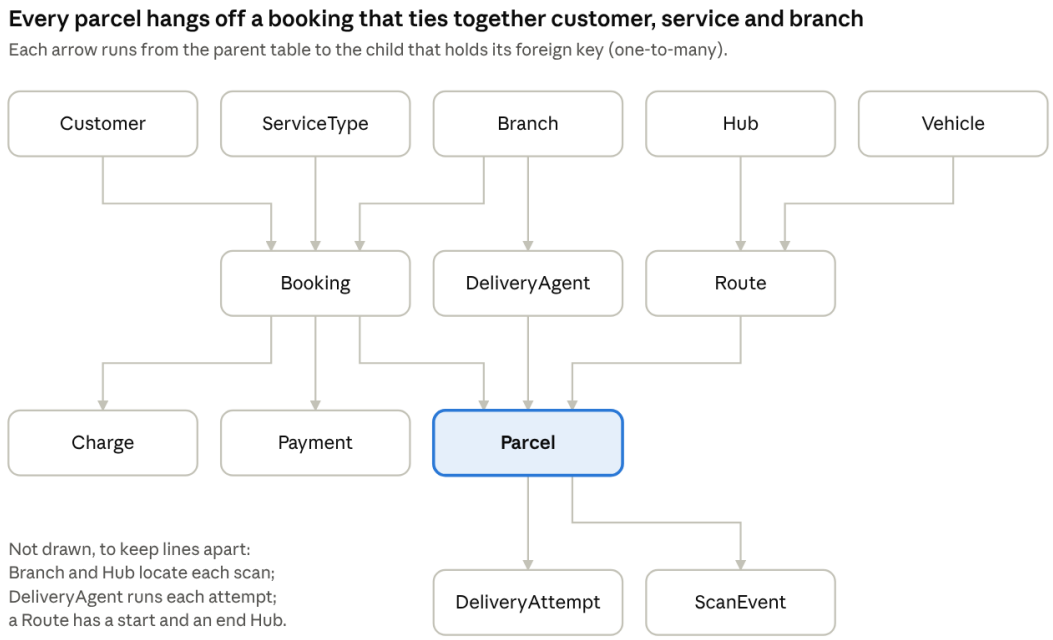


Fig. 1. ER diagram of the Courier Parcel Booking and Tracking System (13 entities, one-to-many links).

The model has 13 entities. Parcel is the centre: it belongs to one booking, may be assigned to one agent and one route, and collects many scan events and delivery attempts.

Relationships and Cardinality

- **Customer – Booking (1:N):** a customer makes many bookings; each booking has exactly one customer.
- **ServiceType – Booking (1:N)** and **Branch – Booking (1:N):** each booking uses one service and starts at one origin branch.
- **Booking – Parcel, Charge, Payment (1:N each):** a booking can hold several parcels, itemized charges and part-payments.
- **Parcel – ScanEvent (1:N):** scans are ordered by `seq_no`; each scan happens at a branch or a hub.
- **Parcel – DeliveryAttempt (1:N):** at most three attempts per parcel.
- **Branch – DeliveryAgent (1:N)** and **DeliveryAgent – Parcel (1:N):** an agent belongs to a branch and delivers many parcels.
- **Hub – Route (1:N, twice)** and **Vehicle – Route (1:N):** a route runs from one hub to another on one vehicle; a route carries many parcels.

IV. Relational Schema

The ER model maps to 13 relations in `courier_db`. Every table has an auto-increment primary key; foreign keys enforce the relationships.

Table	Primary key	Foreign keys	Key constraints
Customer	cust_id	none	phone UNIQUE and NOT NULL; address NOT NULL
Branch	branch_id	none	branch_name, city NOT NULL

Table	Primary key	Foreign keys	Key constraints
Hub	hub_id	none	hub_name, city NOT NULL
ServiceType	service_id	none	service_name UNIQUE; base_charge, per_kg_charge, max_days > 0
DeliveryAgent	agent_id	branch_id → Branch	phone UNIQUE
Vehicle	vehicle_id	none	reg_no UNIQUE; capacity_kg > 0
Route	route_id	from_hub_id, to_hub_id → Hub; vehicle_id → Vehicle	distance_km > 0
Booking	booking_id	cust_id → Customer; service_id → ServiceType; origin_branch_id → Branch	booking_date DEFAULT now
Parcel	parcel_id	booking_id → Booking; route_id → Route; agent_id → DeliveryAgent	tracking_no UNIQUE; weight_kg > 0; status ENUM DEFAULT 'BOOKED'
ScanEvent	scan_id	parcel_id → Parcel; branch_id → Branch; hub_id → Hub	UNIQUE (parcel_id, seq_no); branch or hub must be set
DeliveryAttempt	attempt_id	parcel_id → Parcel; agent_id → DeliveryAgent	UNIQUE (parcel_id, attempt_no); attempt_no 1 to 3
Charge	charge_id	booking_id → Booking	amount > 0
Payment	payment_id	booking_id → Booking	amount > 0; method ENUM (CASH, UPI, CARD)

Indexes

Three secondary indexes speed up the most common lookups: `idx_parcel_status` on Parcel(status), `idx_scan_time` on ScanEvent(scan_time) and `idx_booking_cust` on Booking(cust_id).

V. Normalization

Every table is in third normal form. The example below follows one booking record from an unnormalized layout to 3NF.

Unnormalized form (UNF)

The ScanEvents column holds several values in one cell.

BookingID	CustomerName	ServiceName	BaseRate	ScanEvents
B101	Ravi	Express	50	Booked, In Transit, Delivered
B102	Anu	Standard	30	Booked, In Transit

First normal form (1NF)

Every value is atomic, one row per scan. The key becomes (BookingID, ScanType).

BookingID (PK)	ScanType (PK)	CustomerName	ServiceName	BaseRate

BookingID (PK)	ScanType (PK)	CustomerName	ServiceName	BaseRate
B101	Booked	Ravi	Express	50
B101	In Transit	Ravi	Express	50
B101	Delivered	Ravi	Express	50
B102	Booked	Anu	Standard	30
B102	In Transit	Anu	Standard	30

Second normal form (2NF)

CustomerName, ServiceName and BaseRate depend only on BookingID, not on the full key, so they move to BOOKING. SCAN_EVENT keeps (BookingID, ScanType).

BOOKING: BookingID (PK)	CustomerName	ServiceName	BaseRate
B101	Ravi	Express	50
B102	Anu	Standard	30

Third normal form (3NF)

In BOOKING, BookingID → CustomerID → CustomerName and BookingID → ServiceID → ServiceName, BaseRate are transitive dependencies. They are split into CUSTOMER and SERVICE_TYPE, so each non-key attribute depends only on its own table's key.

CUSTOMER: CustomerID (PK)	CustomerName
C01	Ravi
C02	Anu

SERVICE_TYPE: ServiceID (PK)	ServiceName	BaseRate
S1	Express	50
S2	Standard	30

BOOKING: BookingID (PK)	CustomerID (FK)	ServiceID (FK)
B101	C01	S1
B102	C02	S2

No redundancy remains, so insert, update and delete anomalies are removed. Lookup data (service rates, branch details, hubs) lives in its own tables.

VI. SQL Implementation

The whole database is built by one script, `courier_db.sql`, which drops and recreates `courier_db`, creates the tables, indexes, triggers and views, and loads sample data.

DDL: Database and Table Creation

Parent tables are created first so that foreign keys can reference them. The Booking, Parcel and ScanEvent tables show the main constraints.

```
DROP DATABASE IF EXISTS courier_db;
CREATE DATABASE courier_db;
USE courier_db;


CREATE TABLE Booking (
  booking_id         INT PRIMARY KEY AUTO_INCREMENT,
  cust_id            INT NOT NULL,
  service_id         INT NOT NULL,
  origin_branch_id   INT NOT NULL,
  dest_city          VARCHAR(40) NOT NULL,
  booking_date       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expected_delivery  DATE NOT NULL,
  FOREIGN KEY (cust_id)          REFERENCES Customer(cust_id),
  FOREIGN KEY (service_id)       REFERENCES ServiceType(service_id),
  FOREIGN KEY (origin_branch_id) REFERENCES Branch(branch_id)
);


CREATE TABLE Parcel (
  parcel_id    INT PRIMARY KEY AUTO_INCREMENT,
  booking_id   INT NOT NULL,
  tracking_no  VARCHAR(20) NOT NULL UNIQUE,
  weight_kg    DECIMAL(6,2) NOT NULL CHECK (weight_kg > 0),
  description  VARCHAR(60),
  status       ENUM('BOOKED','IN_TRANSIT','OUT_FOR_DELIVERY','DELIVERED','FAILED')
               NOT NULL DEFAULT 'BOOKED',
  route_id     INT,
  agent_id     INT,
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
  FOREIGN KEY (route_id)   REFERENCES Route(route_id),
  FOREIGN KEY (agent_id)   REFERENCES DeliveryAgent(agent_id)
);


CREATE TABLE ScanEvent (
  scan_id    INT PRIMARY KEY AUTO_INCREMENT,
  parcel_id  INT NOT NULL,
  seq_no     INT NOT NULL CHECK (seq_no > 0),
  branch_id  INT,
  hub_id     INT,
  scan_type  ENUM('BOOKED','PICKED_UP','AT_HUB','OUT_FOR_DELIVERY',
                  'DELIVERED','FAILED_ATTEMPT') NOT NULL,
  scan_time  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (parcel_id, seq_no),
  CHECK (branch_id IS NOT NULL OR hub_id IS NOT NULL),
  FOREIGN KEY (parcel_id) REFERENCES Parcel(parcel_id),
  FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),
  FOREIGN KEY (hub_id)    REFERENCES Hub(hub_id)
);


CREATE INDEX idx_parcel_status ON Parcel(status);
CREATE INDEX idx_scan_time     ON ScanEvent(scan_time);
CREATE INDEX idx_booking_cust  ON Booking(cust_id);
```

`SHOW TABLES` after running the script returns 13 rows: booking, branch, charge, customer, deliveryagent, deliveryattempt, hub, parcel, payment, route, scanevent, servicetype and vehicle.

DML: Sample Data

The script inserts realistic data for Miryalaguda, Hyderabad, Vijayawada and Chennai, so every table has rows to query.

Table	Rows	Sample

Table	Rows	Sample
Customer	5	Ramesh Kumar, Sneha Reddy, Arjun Rao, Priya Sharma, Kiran Babu
Branch / Hub	3 / 3	Miryalaguda, Hyderabad Central, Vijayawada / Hyderabad, Vijayawada, Chennai
ServiceType	3	Standard (40 + 10/kg, 5 days), Express (80 + 20/kg, 2 days), Same Day (150 + 35/kg, 1 day)
DeliveryAgent / Vehicle / Route	3 / 3 / 3	Ravi Kumar, Suresh Reddy, Anil Varma / 1 truck, 2 vans / Hyderabad–Vijayawada 275 km, Vijayawada–Chennai 450 km, Hyderabad–Chennai 630 km
Booking / Parcel	6 / 6	TRK1001 to TRK1006
Charge / Payment	12 / 5	base charge and weight charge per booking
ScanEvent / DeliveryAttempt	23 / 3	full journeys for TRK1001 to TRK1005

```
INSERT INTO Customer (cust_id, cust_name, phone, email, address) VALUES
(1,'Ramesh Kumar','9000000001','ramesh@mail.com','Miryalaguda'),
(2,'Sneha Reddy','9000000002','sneha@mail.com','Miryalaguda');

INSERT INTO Parcel (parcel_id, booking_id, tracking_no, weight_kg,
                    description, route_id, agent_id) VALUES
(1,1,'TRK1001',2.50,'Documents',1,1),
(2,2,'TRK1002',5.00,'Books',1,2);

INSERT INTO ScanEvent (parcel_id, seq_no, branch_id, hub_id,
                       scan_type, scan_time) VALUES
(1,1,1,NULL,'BOOKED','2026-10-01 10:05:00'),
(1,2,1,NULL,'PICKED_UP','2026-10-01 16:00:00'),
(1,3,NULL,1,'AT_HUB','2026-10-02 09:00:00');
```

Parcel status is never typed in by hand: the scan trigger sets it from the latest scan (see Section VIII).

Output of `SELECT * FROM Parcel` after loading the data. Statuses were set by the triggers, not typed in:

parcel_id	tracking_no	weight_kg	description	status	route_id	agent_id
1	TRK1001	2.50	Documents	DELIVERED	1	1
2	TRK1002	5.00	Books	FAILED	1	2
3	TRK1003	10.00	Electronics	IN_TRANSIT	3	3
4	TRK1004	1.20	Gift	DELIVERED	1	3
5	TRK1005	7.50	Clothes	IN_TRANSIT	1	2
6	TRK1006	3.00	Medicines	BOOKED	3	1

VII. SQL Queries

The queries cover joins, aggregates and subqueries. Results below match the output of running the script in MySQL.

Q1. Track a parcel (3-table join)

```
SELECT p.tracking_no, p.status, c.cust_name, b.dest_city, b.expected_delivery
FROM Parcel p
JOIN Booking b  ON b.booking_id = p.booking_id
JOIN Customer c ON c.cust_id = b.cust_id
WHERE p.tracking_no = 'TRK1002';
```

Result: TRK1002, FAILED, Sneha Reddy, Vijayawada, 2026-10-04.

Q2. Scan history of a parcel (join with LEFT JOINs)

```
SELECT p.tracking_no, s.seq_no, s.scan_type,
       COALESCE(br.branch_name, h.hub_name) AS location, s.scan_time
FROM ScanEvent s
JOIN Parcel p ON p.parcel_id = s.parcel_id
LEFT JOIN Branch br ON br.branch_id = s.branch_id
LEFT JOIN Hub h     ON h.hub_id = s.hub_id
WHERE p.tracking_no = 'TRK1002'
ORDER BY s.seq_no;
```

Result: six rows, from BOOKED at Miryalaguda Branch to FAILED_ATTEMPT at Vijayawada Branch. `COALESCE` shows the branch name or, for hub scans, the hub name.

Q3. Charge calculation

```
SELECT p.tracking_no, p.weight_kg, st.service_name,
       st.base_charge + st.per_kg_charge * p.weight_kg AS calculated_charge
FROM Parcel p
JOIN Booking b      ON b.booking_id = p.booking_id
JOIN ServiceType st ON st.service_id = b.service_id;
```

Tracking no.	Service	Weight (kg)	Charge (₹)
TRK1001	Standard	2.50	65
TRK1002	Express	5.00	180
TRK1003	Standard	10.00	140
TRK1004	Same Day	1.20	192
TRK1005	Standard	7.50	115
TRK1006	Express	3.00	140

Q4. Parcels per status (aggregate with GROUP BY)

```
SELECT status, COUNT(*) AS total_parcels FROM Parcel GROUP BY status;
```

Result on the initial data: BOOKED 1, IN_TRANSIT 2, DELIVERED 2, FAILED 1.

Q5. Average and maximum weight

```
SELECT ROUND(AVG(weight_kg),2) AS avg_weight, MAX(weight_kg) AS max_weight FROM Parcel;
```

Result: 4.87 and 10.00.

Q6. Parcels heavier than average (subquery)

```
SELECT tracking_no, weight_kg FROM Parcel
WHERE weight_kg > (SELECT AVG(weight_kg) FROM Parcel);
```

Result: TRK1002 (5.00), TRK1003 (10.00), TRK1005 (7.50).

Q7. Customers with a failed parcel (IN subquery)

```
SELECT cust_name FROM Customer
WHERE cust_id IN (
  SELECT b.cust_id FROM Booking b
  JOIN Parcel p ON p.booking_id = b.booking_id
  WHERE p.status = 'FAILED');
```

Result: Sneha Reddy.

Q8. Bookings billed but not fully paid (correlated subqueries)

```
SELECT b.booking_id,
       (SELECT SUM(amount) FROM Charge  WHERE booking_id = b.booking_id) AS billed,
       IFNULL((SELECT SUM(amount) FROM Payment
               WHERE booking_id = b.booking_id),0) AS paid
FROM Booking b
HAVING billed > paid;
```

Result: booking 5 (115.00 billed, 0.00 paid), booking 2 (180.00, 100.00) and booking 6 (140.00, 70.00).

VIII. Views, Triggers and Transactions

Views for Management Reports

Views wrap the report queries so staff can run `SELECT * FROM <view>` without writing joins.

View	What it reports
`v_current_location`	Latest scan of every parcel: status, location and time
`v_delayed_shipments`	Parcels not delivered after their expected date, with days late
`v_branch_load`	Parcels booked per origin branch
`v_failed_attempts`	Failed delivery attempts per agent
`v_revenue`	Amount billed and amount collected per service type

```
CREATE VIEW v_current_location AS
SELECT p.tracking_no, p.status, s.scan_type,
       COALESCE(br.branch_name, h.hub_name) AS location, s.scan_time
FROM Parcel p
JOIN ScanEvent s ON s.parcel_id = p.parcel_id
LEFT JOIN Branch br ON br.branch_id = s.branch_id
LEFT JOIN Hub h     ON h.hub_id = s.hub_id
WHERE s.seq_no = (SELECT MAX(seq_no) FROM ScanEvent
                  WHERE parcel_id = p.parcel_id);


CREATE VIEW v_delayed_shipments AS
SELECT p.tracking_no, p.status, b.expected_delivery,
       DATEDIFF(CURDATE(), b.expected_delivery) AS days_late
FROM Parcel p
JOIN Booking b ON b.booking_id = p.booking_id
WHERE p.status <> 'DELIVERED' AND b.expected_delivery < CURDATE();
```

Route performance is a query that joins Route, Parcel, Booking and the DELIVERED scan to give the average hours from booking to delivery per route.

```
SELECT r.route_name, COUNT(p.parcel_id) AS parcels,
       ROUND(AVG(TIMESTAMPDIFF(HOUR, bk.booking_date, d.scan_time)),1)
         AS avg_hours_to_deliver
FROM Route r
JOIN Parcel p   ON p.route_id = r.route_id
JOIN Booking bk ON bk.booking_id = p.booking_id
```

```
LEFT JOIN ScanEvent d ON d.parcel_id = p.parcel_id AND d.scan_type = 'DELIVERED'
GROUP BY r.route_id, r.route_name;
```

Output: Hyderabad-Vijayawada has 4 parcels with an average of 50.0 hours to deliver; Hyderabad-Chennai has 2 parcels and NULL, because none of them is delivered yet.

Report Outputs on the Sample Data

`SELECT * FROM v_current_location`:

tracking_no	status	scan_type	location	scan_time
TRK1001	DELIVERED	DELIVERED	Hyderabad Central	2026-10-05 12:30:00
TRK1002	FAILED	FAILED_ATTEMPT	Vijayawada Branch	2026-10-06 14:00:00
TRK1003	IN_TRANSIT	AT_HUB	Chennai Hub	2026-10-05 22:00:00
TRK1004	DELIVERED	DELIVERED	Hyderabad Central	2026-10-05 16:20:00
TRK1005	IN_TRANSIT	AT_HUB	Vijayawada Hub	2026-10-07 20:00:00
TRK1006	BOOKED	BOOKED	Miryalaguda Branch	2026-10-07 12:10:00

`SELECT * FROM v_delayed_shipments` (run on 11 Oct 2026; the result depends on the current date):

tracking_no	status	expected_delivery	days_late
TRK1002	FAILED	2026-10-04	7
TRK1003	IN_TRANSIT	2026-10-08	3

`SELECT * FROM v_branch_load`, `v_failed_attempts` and `v_revenue`:

branch_name	parcels_booked
Miryalaguda Branch	3
Hyderabad Central	2
Vijayawada Branch	1

agent_name	failed_attempts
Suresh Reddy	1

service_name	billed (₹)	collected (₹)
Express	320.00	170.00
Same Day	192.00	192.00
Standard	320.00	205.00

Triggers: Business Rules Inside the Database

Trigger	When	Rule it enforces

Trigger	When	Rule it enforces
trg_scan_before_insert	before a scan is inserted	Rejects a scan if the parcel is already DELIVERED ('No scans allowed after delivery') or if seq_no is not the last number plus one ('Invalid scan sequence')
trg_scan_after_insert	after a scan is inserted	Updates Parcel.status from the scan type: PICKED_UP and AT_HUB give IN_TRANSIT; OUT_FOR_DELIVERY and DELIVERED map to themselves; a failed attempt gives FAILED
trg_payment_before_insert	before a payment is inserted	Rejects a payment if total paid would exceed total charges ('Payment exceeds total charge')

```
CREATE TRIGGER trg_payment_before_insert
BEFORE INSERT ON Payment
FOR EACH ROW
BEGIN
  DECLARE total_charge DECIMAL(10,2);
  DECLARE total_paid   DECIMAL(10,2);
  SELECT IFNULL(SUM(amount), 0) INTO total_charge
    FROM Charge  WHERE booking_id = NEW.booking_id;
  SELECT IFNULL(SUM(amount), 0) INTO total_paid
    FROM Payment WHERE booking_id = NEW.booking_id;
  IF total_paid + NEW.amount > total_charge THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Payment exceeds total charge';
  END IF;
END
```

Transactions

Booking a parcel touches four tables, so it runs as one transaction: either the booking, parcel, charges, first scan and payment are all saved, or none are.

```
START TRANSACTION;
INSERT INTO Booking (booking_id, cust_id, service_id, origin_branch_id,
                     dest_city, expected_delivery)
VALUES (7, 2, 1, 1, 'Hyderabad', '2026-12-31');
INSERT INTO Parcel (parcel_id, booking_id, tracking_no, weight_kg,
                    description, route_id, agent_id)
VALUES (7, 7, 'TRK1007', 4.00, 'Shoes', 1, 1);
INSERT INTO Charge (booking_id, description, amount)
VALUES (7,'Base charge',40),(7,'Weight charge',40);
INSERT INTO ScanEvent (parcel_id, seq_no, branch_id, scan_type)
VALUES (7, 1, 1, 'BOOKED');
INSERT INTO Payment (booking_id, amount, method) VALUES (7, 80, 'UPI');
COMMIT;
```

A second transaction updates TRK1007's description to 'WRONG' and then runs ROLLBACK; the follow-up SELECT shows the original description, which proves the change was undone.

IX. Testing: Constraints and Error Handling

The script ends with valid operations and deliberately invalid ones. MySQL rejected every invalid statement; the observed output is shown in the fourth column.

#	Test case	Statement	Observed result	Pass
1	Book a parcel in one transaction	Booking, Parcel, Charge, ScanEvent, Payment for TRK1007	COMMIT succeeded; v_current_location shows TRK1007 as BOOKED at Miryalaguda Branch	Yes

#	Test case	Statement	Observed result	Pass
2	Rollback	Update description to ‘WRONG’, then ROLLBACK	Description is still ‘Shoes’	Yes
3	Valid scans for TRK1006	PICKED_UP, then AT_HUB (seq 2 and 3)	Accepted; status IN_TRANSIT at Hyderabad Hub	Yes
4	Failed then successful delivery of TRK1005	OUT_FOR_DELIVERY, FAILED_ATTEMPT (‘Address locked’), retry, DELIVERED; attempts 1 and 2	Accepted; final status DELIVERED	Yes
5	Duplicate tracking number	Insert Parcel with ‘TRK1001’	ERROR 1062: Duplicate entry ‘TRK1001’ for key ‘parcel.tracking_no’	Yes
6	Negative weight	Insert Parcel with weight -3.0	ERROR 3819: Check constraint ‘parcel_chk_1’ is violated	Yes
7	Negative charge	Insert Charge with amount -10	ERROR 3819: Check constraint ‘charge_chk_1’ is violated	Yes
8	Scan after delivery	Scan seq 6 for delivered TRK1001	ERROR 1644: No scans allowed after delivery	Yes
9	Scan out of sequence	Scan seq 9 for TRK1003	ERROR 1644: Invalid scan sequence	Yes
10	Overpayment	Payment of 50 on fully paid booking 1	ERROR 1644: Payment exceeds total charge	Yes
11	Unknown customer	Booking for cust_id 99	ERROR 1452: foreign key constraint fails (booking_ibfk_1)	Yes
12	Update a customer	Change Sneha Reddy’s address to Nalgonda	1 row changed; address shows Nalgonda	Yes
13	Insert then delete a customer	Add and remove ‘Temp User’	Both succeeded, 1 row each	Yes
14	Delete a referenced customer	Delete cust_id 1	ERROR 1451: cannot delete a parent row (booking_ibfk_1)	Yes

Tests 5 to 11 and 14 show that integrity is enforced by the database itself, whichever application sends the statement.

X. Live Simulation

A web simulation of the system is hosted at https://courier-tracking-system-x7be.onrender.com/. It is one page whose sections map onto the database operations described in this report. Each action in the page ends up as SQL against the courier schema, so a change made on screen can be checked in MySQL afterwards (insert, view, delete, verify).

Screen section	What the user does	Database idea behind it
All parcels	Views every parcel	SELECT with joins on Parcel, Booking, Customer
Book a parcel	Picks customer, service, origin branch, destination city, weight and description, then books	Transaction over Booking, Parcel and Charge; charge = base + per-kg rate × weight

Screen section	What the user does	Database idea behind it
Track a parcel	Enters a tracking number	Scan history join and `v_current_location`
Add tracking scan	Adds PICKED_UP, AT_HUB, OUT_FOR_DELIVERY, DELIVERED or FAILED_ATTEMPT with a location and a reason for failures	Scan triggers check the sequence and update parcel status
Make a payment	Pays by CASH, UPI or CARD against a tracking number	Payment trigger blocks overpayment
Assign delivery agent	Chooses an agent for a tracking number	UPDATE of Parcel.agent_id (foreign key to DeliveryAgent)
Customers	Adds a customer with name, phone, email and address	UNIQUE phone; delete blocked while bookings exist
Reports	Runs one of six reports: current parcel location, delayed shipments, branch load, route performance, failed attempts, revenue	The views and queries in Sections VII and VIII

Typical Walkthrough

1. Add or pick a customer, then book a parcel with the Express service to Vijayawada.
2. Note the tracking number and track it: the status is BOOKED.
3. Add scans in order: PICKED_UP, AT_HUB, OUT_FOR_DELIVERY. The status follows each scan.
4. Add FAILED_ATTEMPT with a reason such as ‘Customer not available’, then retry and add DELIVERED.
5. Pay part of the charge, then try to pay more than the balance and watch the rejection.
6. Open Reports to see the parcel in revenue, branch load and failed-attempt reports.

Screenshots of each screen, before and after each action, can be placed under this section.

XI. Conclusion and Future Scope

The project delivers a 3NF relational database in MySQL that tracks a parcel from booking to delivery and payment. Primary keys, foreign keys, unique and CHECK constraints protect the data; three triggers keep scan order, parcel status and payments consistent; views give management ready-made reports; and transactions make a booking all-or-nothing. The web simulation shows these rules working through a real interface.

Future enhancements:

- User login and role-based access for customers, branch staff, agents and admins.
- SMS or email notifications on each scan.
- Online payment gateway.
- Live vehicle tracking and automatic route selection.
- Dashboards and charts for the report views.

Contribution of Team Members

Member	Part of the presentation
Akshaya	Team and project introduction; problem, objectives, scope and users; live demo
Srinidhi	ER diagram, relational schema and normalization to 3NF

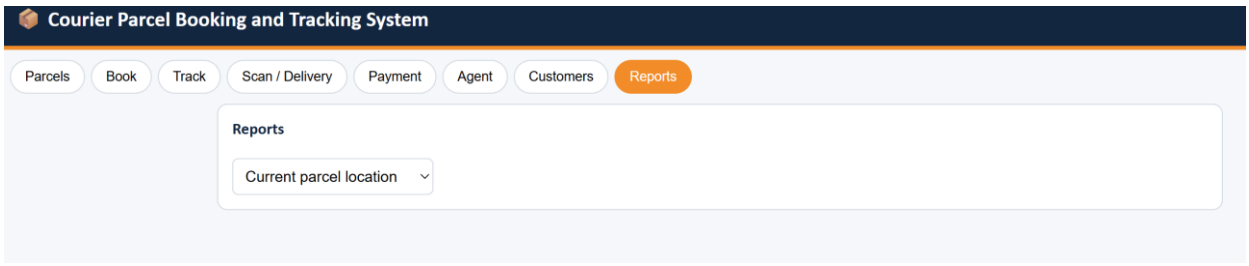
Member	Part of the presentation
Sahasra	Database and table creation, inserting records, SELECT queries

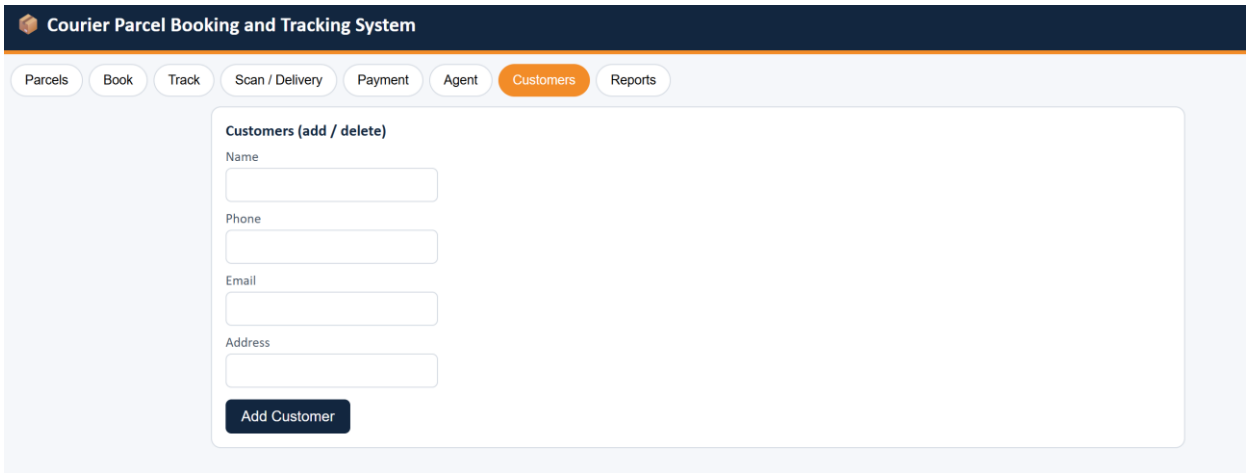
Everyone on the team is prepared to explain any part of the project.

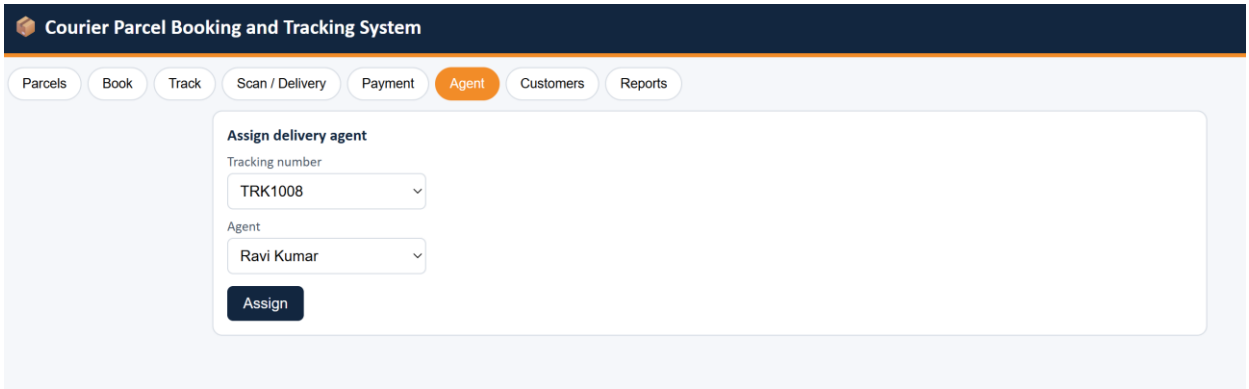
References

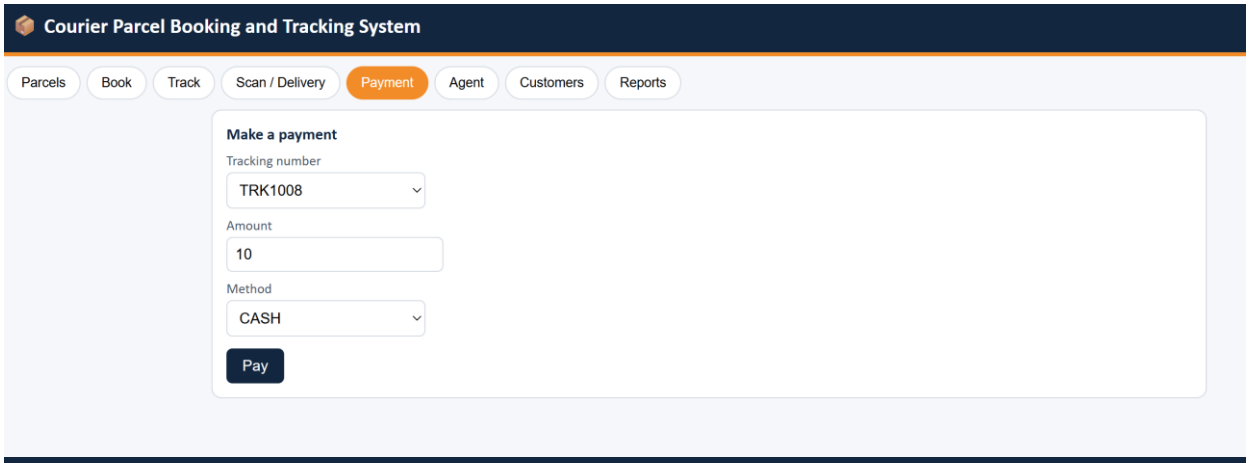
1. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
2. Oracle Corporation, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
3. Courier Parcel Booking and Tracking System, live simulation. https://courier-tracking-system-x7be.onrender.com/

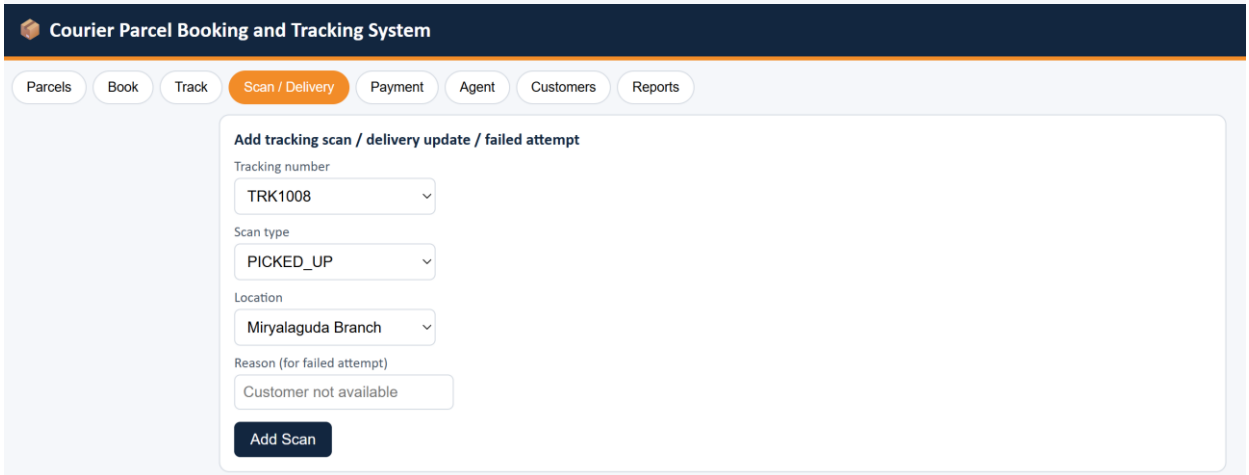
Simulation

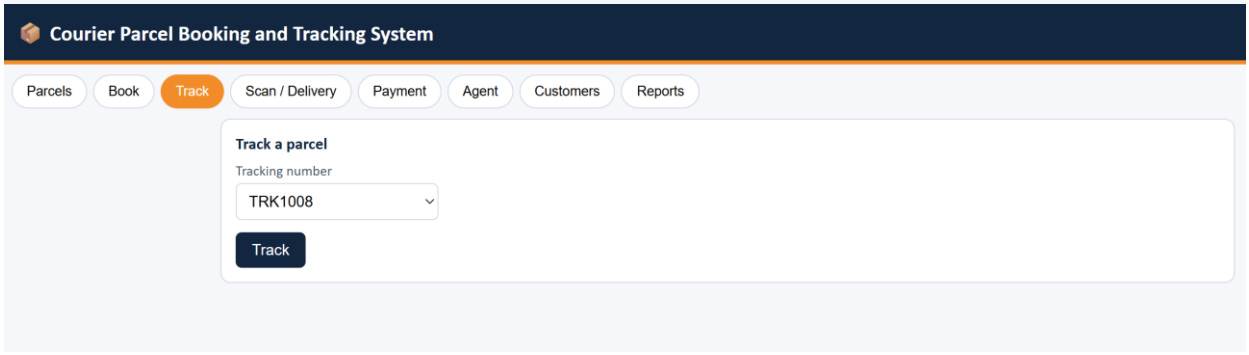


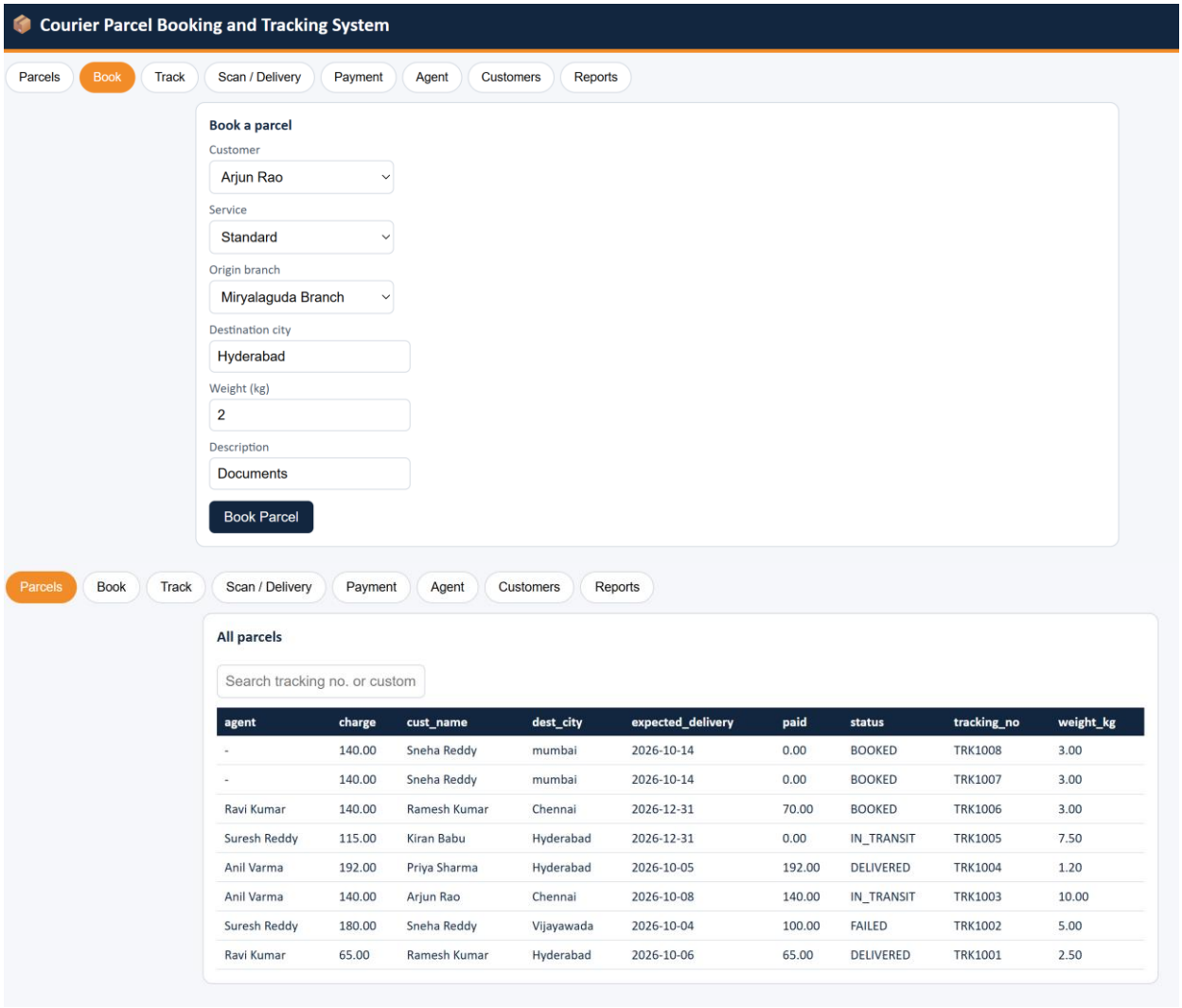





Courier Parcel Booking and Tracking System
Parcels
Book
Track
Scan / Delivery
Payment
Agent
Customers
Reports
Make a payment
Tracking number
TRK1008
Amount
10
Method
CASH
Pay


Courier Parcel Booking and Tracking System
Parcels
Book
Track
Scan / Delivery
Payment
Agent
Customers
Reports
Add tracking scan / delivery update / failed attempt
Tracking number
TRK1008
Scan type
PICKED_UP
Location
Miryalaguda Branch
Reason (for failed attempt)
Customer not available
Add Scan


Courier Parcel Booking and Tracking System
Parcels
Book
Track
Scan / Delivery
Payment
Agent
Customers
Reports
Track a parcel
Tracking number
TRK1008
Track



agent	charge	cust_name	dest_city	expected_delivery	paid	status	tracking_no	weight_kg
-	140.00	Sneha Reddy	mumbai	2026-10-14	0.00	BOOKED	TRK1008	3.00
-	140.00	Sneha Reddy	mumbai	2026-10-14	0.00	BOOKED	TRK1007	3.00
Ravi Kumar	140.00	Ramesh Kumar	Chennai	2026-12-31	70.00	BOOKED	TRK1006	3.00
Suresh Reddy	115.00	Kiran Babu	Hyderabad	2026-12-31	0.00	IN_TRANSIT	TRK1005	7.50
Anil Varma	192.00	Priya Sharma	Hyderabad	2026-10-05	192.00	DELIVERED	TRK1004	1.20
Anil Varma	140.00	Arjun Rao	Chennai	2026-10-08	140.00	IN_TRANSIT	TRK1003	10.00
Suresh Reddy	180.00	Sneha Reddy	Vijayawada	2026-10-04	100.00	FAILED	TRK1002	5.00
Ravi Kumar	65.00	Ramesh Kumar	Hyderabad	2026-10-06	65.00	DELIVERED	TRK1001	2.50